banQi Consignado — Fluxo Completo de Integração

banQi

Maio 2026

Fluxo Completo — Do “Oi” ao Empréstimo Aprovado

Simulação passo a passo de todos os payloads trocados entre WhatsApp, Lambda, AgentCore e banQi API.

ETAPA 0 — Cliente envia “Oi” no WhatsApp

Meta → Lambda (POST /webhook)

```json
{
  "object": "whatsapp_business_account",
  "entry": [{
    "id": "123456789",
    "changes": [{
      "value": {
        "messaging_product": "whatsapp",
        "metadata": { "phone_number_id": "987654321" },
        "messages": [{
          "id": "wamid.ABC123XYZ",
          "from": "5511999990000",
          "timestamp": "1747180800",
          "type": "text",
          "text": { "body": "Oi" }
        }]
      }
    }]
  }]
}
```

Headers enviados pela Meta:

```
X-Hub-Signature-256: sha256=a1b2c3d4e5f6...   ← HMAC-SHA256 do body
Content-Type: application/json
```

ETAPA 0.1 — Lambda valida e deduplica

```
handler.py executa em sequência:

1. validate_webhook_signature(body, "sha256=a1b2c3d4...", APP_SECRET)
   → hmac.compare_digest(esperado, recebido) == True ✓

2. DynamoDB.put_item(
     TableName: "banqi-dedup",
     Item: { message_id: "wamid.ABC123XYZ", ttl: now+120 },
     ConditionExpression: "attribute_not_exists(message_id)"
   )
   → Sucesso — mensagem nova ✓

3. Extrai: phone="5511999990000", text="Oi"
```

ETAPA 0.2 — Lambda chama AgentCore Runtime

Lambda → AgentCore Runtime

```python
# agentcore_client.py
client.invoke_agent_runtime(
    agentRuntimeArn="arn:aws:bedrock-agentcore:us-east-1:...",
    qualifier="DEFAULT",
    payload={
        "prompt": "Oi",
        "user_id": "5511999990000",
        "session_id": "session-5511999990000-1747180800"
    }
)
```

ETAPA 0.3 — Supervisor carrega memória LTM

O Supervisor (Claude Sonnet 4.6) busca memória do usuário:

```
Namespace consultado: users/5511999990000/consignado
Resultado: (vazio — primeiro contato)

current_step = 0   → usuário novo
consent_accepted = false
```

Decisão do Supervisor: mensagem "Oi" → rota para `consignado_assistant`

Contexto injetado na delegação:

```
Usuário: Oi

Contexto do usuário (memória LTM):
- current_step: 0 (novo usuário)
- Nenhum dado coletado ainda

Inicie o fluxo pelo Etapa 1 (Consentimento LGPD).
```

ETAPA 1 — Consentimento LGPD

Consignado Agent → banQi API

```
POST https://api.banqi.com.br/consignado/v1/consent-term
Headers:
  x-whatsapp-phone: 5511999990000
  x-document: (ainda não coletado)
  x-partner: banqi-wpp
  Authorization: Bearer eyJhbGciOiJSUzI1NiJ9...
```

banQi API → Agent:

```
{
  "status": 202,
  "termId": "term_abc123",
  "summary": "Autorizo o banQi a consultar meu histórico no INSS...",
  "version": "2.0"
}
```

Agent → Cliente (via WhatsApp):

```
Olá! Sou o assistente de empréstimo consignado banQi 😊

Para continuar, preciso da sua autorização:

📋 *Termo de Consentimento banQi*
Autorizo consulta ao INSS, análise de margem consignável e
uso dos dados para esta contratação.

Você aceita os termos de uso e política de privacidade do banQi?
```

Lambda → Meta Graph API (enviar mensagem):

```
POST https://graph.facebook.com/v17.0/987654321/messages
Authorization: Bearer EAABs...whatsapp_token
Content-Type: application/json

{
  "messaging_product": "whatsapp",
  "to": "5511999990000",
  "type": "text",
  "text": {
    "body": "Olá! Sou o assistente de empréstimo consignado banQi 😊\n\n..."
  }
}
```

Cliente responde: “Sim, aceito”

Agent → banQi API:

```
POST https://api.banqi.com.br/consignado/v1/consent-term/accept
Headers: x-whatsapp-phone, x-partner: banqi-wpp
Body: { "termId": "term_abc123", "accepted": true }
```

banQi → Agent:

```
{ "status": 200, "message": "Consent accepted" }
```

Memória LTM atualizada:

```
{
  "consent_accepted": true,
  "current_step": 2
}
```

ETAPA 2 — Simulação

Agent → Cliente:

```
Ótimo! Agora vamos simular seu empréstimo. 💰

Qual valor você precisa?
(mínimo R$ 500, máximo R$ 50.000)
```

Cliente: `"Quero 10000"`

```
validate_loan_amount(10000) → True ✓
valor_solicitado = 10000.00
```

Agent → Cliente:

```
Em quantas parcelas?
(opções: 12, 24, 36, 48 ou 60)
```

Cliente: `"36"`

Agent → banQi API:

```
POST https://api.banqi.com.br/consignado/v1/simulations
Headers:
  x-whatsapp-phone: 5511999990000
  x-document: (CPF do cliente)
  x-partner: banqi-wpp

Body:
{
  "amount": 10000.00,
  "installments": 36,
  "phone": "5511999990000"
}
```

banQi → Agent (HTTP 200 — simulação imediata):

```
{
  "simulationId": "sim_XYZ789",
  "amount": 10000.00,
  "installments": 36,
  "installmentValue": 385.20,
  "interestRate": 1.89,
  "cet": 25.43,
  "estimatedDepositDate": "2026-05-20",
  "status": "READY"
}
```

Agent → Cliente:

```
📊 *Simulação banQi Consignado:*

Valor: R$ 10.000,00
Parcelas: 36 x R$ 385,20
Taxa: 1,89% ao mês
CET: 25,43% ao ano
Depósito estimado: 20/05/2026

Quer contratar com essas condições?
```

Memória LTM atualizada:

```
{
  "simulation_id": "sim_XYZ789",
  "valor_solicitado": 10000.00,
  "parcelas_escolhidas": 36,
  "current_step": 3
}
```

ETAPA 3 — Dados Bancários (um campo por mensagem)

```
Agent:   "Qual é o seu banco? (código de 3 dígitos ou nome)"
Cliente: "Itaú"
         → validate_bank_code("Itaú") → "341" ✓

Agent:   "Qual é o número da sua agência?"
Cliente: "1234"

Agent:   "Qual é o número da sua conta (com dígito verificador)?"
Cliente: "56789-0"
         → exibido mascarado: ****89-0

Agent:   "Qual é o tipo da conta?"
         (Corrente, Poupança, Pagamento, Salário)
Cliente: "Corrente"
         → normalize_account_type("Corrente") → "CHECKING"
```

Memória LTM atualizada:

```
{
  "banco": "341",
  "agencia": "1234",
  "conta": "56789-0",
  "tipo_conta": "CHECKING",
  "current_step": 4
}
```

ETAPA 4 — Criação da Proposta

Agent → banQi API:

```
POST https://api.banqi.com.br/consignado/v1/proposals
Headers:
  x-whatsapp-phone: 5511999990000
  x-document: (CPF do cliente)
  x-partner: banqi-wpp

Body:
{
  "simulationId": "sim_XYZ789",
  "bankAccount": {
    "bankCode": "341",
    "agency": "1234",
    "account": "56789-0",
    "accountType": "CHECKING"
  }
}
```

banQi → Agent (HTTP 202 — processando de forma assíncrona):

```
{
  "proposalId": "prop_QRS456",
  "status": "PENDING"
}
```

Alguns segundos depois — banQi envia webhook assíncrono:

```
POST https://api.gw.amazonaws.com/.../webhook/banqi
Headers: X-BanQi-Signature: sha256=def456...

Body:
{
  "event": "PROPOSAL_STATUS_UPDATE",
  "phone": "5511999990000",
  "newStatus": "APPROVED",
  "idProposal": "prop_QRS456"
}
```

Lambda roteia o evento (router.py):

```
_HANDLERS["PROPOSAL_STATUS_UPDATE"](payload)
→ busca session_id no DynamoDB via phone
→ envia mensagem ao cliente
```

Agent → Cliente:

```
Sua proposta foi aprovada! 🎉
Vamos para a validação de identidade.
```

ETAPA 5 — Biometria

Agent → banQi API:

```
POST https://api.banqi.com.br/consignado/v1/biometry/start
Headers: x-whatsapp-phone, x-document, x-partner

Body:
{
  "proposalId": "prop_QRS456",
  "phone": "5511999990000"
}
```

banQi → Agent:

```
{
  "biometryId": "bio_DEF321",
  "link": "https://biometria.banqi.com.br/flow/bio_DEF321",
  "idAntiFraud": "af_789XYZ"
}
```

Agent → Cliente:

```
Para finalizar, precisamos validar sua identidade. 📸

Acesse o link abaixo e tire uma selfie:
https://biometria.banqi.com.br/flow/bio_DEF321

(link válido por 30 minutos)
```

Depois que o cliente faz a selfie — banQi envia webhook:

```
{
  "event": "BIOMETRY_STATUS_UPDATE",
  "phone": "5511999990000",
  "newStatus": "APPROVED",
  "idProposal": "prop_QRS456"
}
```

Agent → Cliente:

```
Biometria aprovada! ✅
Vamos para a etapa final de confirmação.
```

ETAPA 6 — Aceite da Proposta

Agent → Cliente:

```
📋 *Resumo final do seu empréstimo:*

Valor: R$ 10.000,00
Parcelas: 36 x R$ 385,20
Taxa: 1,89% ao mês
Banco: Itaú | Conta: ****89-0 (Corrente)
Depósito: 20/05/2026

Confirma a contratação do empréstimo consignado?
```

Cliente: `"Sim, confirmo"`

Agent → banQi API:

```
POST https://api.banqi.com.br/consignado/v1/proposals/prop_QRS456/accept
Headers: x-whatsapp-phone, x-document, x-partner
```

banQi → Agent (webhook CONTRACT_SIGNED):

```
{
  "event": "CONTRACT_SIGNED",
  "phone": "5511999990000",
  "idProposal": "prop_QRS456",
  "depositDate": "2026-05-20"
}
```

ETAPA 7 — Conclusão

Agent → Cliente:

```
Contrato assinado! 🎊

O depósito de R$ 10.000,00 será realizado em
até 2 dias úteis na sua conta Itaú.

Qualquer dúvida, é só chamar aqui! 😊
```

Memória LTM final:

```json
{
  "current_step": 7,
  "proposal_id": "prop_QRS456",
  "simulation_id": "sim_XYZ789",
  "consent_accepted": true,
  "valor_solicitado": 10000.00,
  "parcelas_escolhidas": 36,
  "banco": "341",
  "agencia": "1234",
  "conta": "56789-0",
  "tipo_conta": "CHECKING"
}
```

Resumo de todos os fluxos de dados

Quem envia	Para quem	Protocolo	O que vai
Meta (WhatsApp)	Lambda	HTTPS POST	Payload JSON + HMAC-SHA256 header
Lambda	AgentCore Runtime	AWS SDK (Bedrock)	Prompt + user_id + session_id
AgentCore	Claude Sonnet 4.6	Bedrock API	System prompt + histórico + memória LTM
Supervisor (Sonnet)	Consignado Agent (Haiku)	Strands Agents-as-Tools	Contexto completo + current_step
Consignado Agent	banQi API	HTTPS via MCP Gateway	Headers obrigatórios + body JSON
banQi API	Lambda	HTTPS POST	Webhooks de status assíncrono
Lambda	Meta Graph API	HTTPS POST	Mensagem de texto formatada

Resumo dos webhooks assíncronos banQi

Evento	Quando dispara	Ação do agente
`CONSENT_TERM_FILE_READY`	PDF do termo disponível	Envia link do PDF
`NO_OFFER_AVAILABLE`	Sem margem consignável	Informa e oferece tentar em 30 dias
`SIMULATION_READY`	Simulação processada	Apresenta resultado ao cliente
`PROPOSAL_CREATED`	Proposta registrada	Confirma criação
`PROPOSAL_STATUS_UPDATE: APPROVED`	Proposta aprovada	Avança para biometria
`PROPOSAL_STATUS_UPDATE: DENIED`	Proposta negada	Oferece nova simulação
`BIOMETRY_STATUS_UPDATE: APPROVED`	Selfie aprovada	Avança para aceite
`BIOMETRY_STATUS_UPDATE: DENIED`	Selfie rejeitada	Solicita nova tentativa
`CONTRACT_SIGNED`	Contrato assinado	Confirma depósito e encerra

Códigos de erro e respostas

Erro da API	HTTP	O que o agente responde
`ELIGIBILITY_REJECTED`	422	“No momento não encontramos oferta. Posso tentar em 30 dias.”
`TOKEN_EXPIRED`	401	Renova token via Cognito silenciosamente e retenta
`INSUFFICIENT_MARGIN`	422	“Seu limite não é suficiente. Quer tentar um valor menor?”
`ALREADY_ACTIVE`	409	“Você já possui um termo ativo. Vamos continuar.”
HTTP 5xx	500-503	“Estamos com instabilidade. Tente novamente em alguns minutos.”
HTTP 422	422	Identifica campo com erro e pede correção específica